DYNAMIC PROGRAMMING PROJECT

Spring 2025 / 2026

The Robot & the Crystals

Algorithms & Data Structures Course

DP Assignment

Problem Summary

A robot travels through N energy stations in a straight line.
From station i it may jump +1 or +2 steps, collecting each station's energy.
It may skip (ignore) the value of at most ONE visited station.
Goal: maximise total energy collected.

Divide & Conquer	Dynamic Programming	Space: O(N)	Time: O(N)

Problem Description

A robot travels along a straight tunnel made of N energy stations. Each station holds an energy crystal with a positive or negative integer value. The robot starts at station 1 and at every step may advance exactly 1 or 2 positions, collecting the crystal of each station it lands on. Once per journey the robot may silently ignore the value of one visited station (its "skip" power). The objective is to find the path that maximises the total collected energy.

Formal Rules

1. The robot starts at station 1 (mandatory first stop).
2. From station i the robot moves to i+1 OR i+2.
3. Every landed station contributes its value (positive or negative).
4. The robot may ignore at most ONE station's value during the trip.
5. The journey ends when the robot moves beyond station N.

Worked Example

N = 6 A = [3, -5, 4, -2, 7, 1]

	i=1	i=2	i=3	i=4	i=5	i=6
Station	1	2	3	4	5	6
Energy	3	-5	4	-2	7	1

Three representative paths:

Path	Stations visited	Raw sum	After skip
1 → 3 → 5 → 6	3, 4, 7, 1	15	15 (no skip needed)
1 → 2 → 4 → 6	3, -5, -2, 1	-3	2 (skip -5)
1 → 2 → 3 → 5	3, -5, 4, 7	9	14 (skip -5)

Optimal Path

Path: 1 → 3 → 5 → 6 Collected: 3 + 4 + 7 + 1 = 15 Skip: not needed Answer: 15

Part 1 — Divide & Conquer

Step 1: Define the Function f

We define the recursive function:

Function Definition

f(i, canSkip) = maximum energy collectible from station i to the end of the tunnel

i — current station index (1-based, 1 ≤ i ≤ N)
canSkip — 1 if the skip power is still available, 0 if already used

Return value: an integer representing the best possible total energy from station i onward.

Step 2: Parameters and Recurrence

Parameters

Parameter	Type	Range	Meaning
i	int	1 … N	Index of the current station (1-based)
canSkip	int	0 or 1	1 = skip power available, 0 = already consumed

Base Case

If i > N, the robot has exited the tunnel. No more energy is collected:

```
f(i, canSkip) = 0     for all i > N
```

Recursive Cases

At station i the robot has two collect-or-skip choices then two movement choices:

```
// Collect station i, skip power unchanged
collect = A[i] + max( f(i+1, canSkip), f(i+2, canSkip) )
```

```
// Skip station i (value = 0), skip power consumed
// Only available when canSkip == 1
skip = max( f(i+1, 0), f(i+2, 0) )

if canSkip == 1:
    f(i, 1) = max(collect, skip)
else:
    f(i, 0) = collect                // must collect; no skip left
```

Important Observation

The 'skip' in f(i,1) means we skip station i itself — not some future station.
Once used, canSkip becomes 0 and all subsequent calls must collect.
The robot always moves +1 or +2 regardless of whether it skips.

Step 3: Recursion Tree (Example N=6)

The tree below shows the first three levels of f(1,1). Each node is labelled f(i, s). Bold branches show the choices made on the optimal path.

```
                        f(1, 1)                              ← root
              /collect            \skip
         f(2,1)+3              f(2,0)   f(3,0)
        /        \
    f(3,1)-5     f(4,1)-5
    /     \
 f(4,1)   f(5,1)   ← continue expanding …

  Optimal path nodes (no skip needed):
  f(1,1) → collect 3 → move+2 → f(3,1)
         → collect 4 → move+2 → f(5,1)
         → collect 7 → move+1 → f(6,1)
         → collect 1 → exit   → 0
  Total: 3 + 4 + 7 + 1 = 15
```

Each node f(i,s) generates at most 4 children (2 moves × collect/skip), but many states are shared — this is the key motivation for memoisation / DP.

Step 4: Divide & Conquer Code (Java 15)

```
import java.util.*;

public class DnCRobot {
    static int[] A;
    static int   N;
```

```
    /* f(i, canSkip):
     *   i        - current station (1-based)
     *   canSkip  - 1 if skip power still available, 0 if used  */
    static int f(int i, int canSkip) {
        if (i > N) return 0;                          // base case: beyond tunnel

        // Option A: collect this station, skip status unchanged
        int collect = A[i] + Math.max(f(i + 1, canSkip),
                                      f(i + 2, canSkip));

        if (canSkip == 0) return collect;             // no skip left → must
collect

        // Option B: ignore this station's value, skip consumed
        int skip = Math.max(f(i + 1, 0),
                            f(i + 2, 0));

        return Math.max(collect, skip);
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        N = sc.nextInt();
        A = new int[N + 3];                          // 1-indexed, 2 guard cells
        for (int i = 1; i <= N; i++) A[i] = sc.nextInt();

        System.out.println(f(1, 1));
    }
}
```

Complexity of the Naive Recursion

Time: O(2^N) — each call spawns up to 4 sub-calls; depth is N.
Space: O(N) — call-stack depth.
Overlapping sub-problems: f(i, s) is recomputed exponentially many times.
Solution: memoisation (top-down) or tabulation (bottom-up DP).

Part 2 — Dynamic Programming

Step 5: DP Table and Dependencies

Table Definition

We use a 2-D table dp[1..N][0..1] where:

dp[i][s]

dp[i][s] = maximum energy the robot can collect
starting at station i and going to the end,
given that the skip power status is s.

s = 1 → skip power still available
s = 0 → skip power already used

Recurrence (bottom-up form)

```
// Base cases (two sentinel columns beyond the array)
dp[i][s] = 0    for all i > N, s ∈ {0,1}

// s = 0 : skip power used; must collect every remaining station
dp[i][0] = A[i] + max( dp[i+1][0], dp[i+2][0] )

// s = 1 : skip power available; choose to collect or to skip this station
dp[i][1] = max(
    A[i] + max( dp[i+1][1], dp[i+2][1] ),   // collect station i
           max( dp[i+1][0], dp[i+2][0] )    // skip station i
)

// Answer
answer = dp[1][1]
```

Dependency Diagram

Each cell dp[i][*] depends on dp[i+1][*] and dp[i+2][*]. The arrows show the direction of dependency:

```
  dp[1][s]  ←  dp[2][s]  ←  dp[3][s]  ← … ←  dp[N][s]  ←  (0)
                    ↑                            ↑
             dp[3][s]                      dp[N+2][s]

  Each cell looks RIGHT (+1) and SKIP-RIGHT (+2).
  Therefore we fill the table from RIGHT to LEFT (i = N down to 1).
```

Filled Table for the Example

A = [3, -5, 4, -2, 7, 1], N = 6 (sentinel columns 7 and 8 hold 0)

i	A[i]	dp[i][0] (skip used)	dp[i][1] (skip available)	Note
i	A[i]	dp[i][0]	dp[i][1]	Best at i
7 (sentinel)	—	0	0	—
6	1	1	1	1
5	7	8	8	8
4	-2	6	**8**	Skip station 4
3	4	12	12	12
2	-5	7	**12**	Skip station 2
1 (start)	3	15	**15**	Answer = 15

Highlighted cells show where the skip decision actually improves the value.

Step 6: Direction of Table Filling

Because dp[i][s] depends on dp[i+1][s] and dp[i+2][s], we must compute larger indices before smaller ones.

Filling Order

Outer loop → i from N down to 1 (right → left)
Inner loop → s from 0 to 1 (order independent within same i)

Sentinel: dp[N+1][*] = dp[N+2][*] = 0 (robot beyond tunnel)

```
for (int i = N; i >= 1; i--) {
    dp[i][0] = A[i] + Math.max(dp[i+1][0], dp[i+2][0]);

    int collectHere = A[i] + Math.max(dp[i+1][1], dp[i+2][1]);
    int skipHere    =        Math.max(dp[i+1][0], dp[i+2][0]);
    dp[i][1] = Math.max(collectHere, skipHere);
}
```

Step 7: Full Dynamic Programming Code (Java 15)

```
import java.util.*;

public class Solution {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int N = sc.nextInt();
        int[] A = new int[N + 3];           // 1-indexed, 2 sentinel slots
        for (int i = 1; i <= N; i++) A[i] = sc.nextInt();

        System.out.println(maxEnergy(N, A));
    }

    // ------------------------------------------------------------------
    // maxEnergy: fills the DP table and returns the optimal value
    // ------------------------------------------------------------------
    static int maxEnergy(int N, int[] A) {
        // dp[i][0] = max energy from station i, skip already used
        // dp[i][1] = max energy from station i, skip still available
        int[][] dp = new int[N + 3][2];    // extra rows act as sentinels (= 0)

        for (int i = N; i >= 1; i--) {
            // Case s=0: no skip left — must collect
            dp[i][0] = A[i] + Math.max(dp[i + 1][0], dp[i + 2][0]);

            // Case s=1: skip available — choose best of collect or skip
            int collectHere = A[i] + Math.max(dp[i + 1][1], dp[i + 2][1]);
            int skipHere    =        Math.max(dp[i + 1][0], dp[i + 2][0]);
            dp[i][1] = Math.max(collectHere, skipHere);
        }

        return dp[1][1];
    }
}
```

Time Complexity	Space Complexity
O (N)	**O (N)**
One pass through N stations. Each cell computed in O(1).	Table of size 2N. Reducible to O(1) with a 6-variable rolling window.

Step 8: Path Reconstruction

To print the actual sequence of moves we store the decision made at each cell — collect/skip and the jump size (+1 or +2) — in a separate choice table, then trace forward from station 1.

```
import java.util.*;

public class SolutionWithPath {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int N = sc.nextInt();
        int[] A = new int[N + 3];
        for (int i = 1; i <= N; i++) A[i] = sc.nextInt();
        solve(N, A);
    }

    static void solve(int N, int[] A) {
        int[][] dp     = new int[N + 3][2];
        // choice[i][s] encodes: bit0 = jump size-1 (0→+1, 1→+2)
        //                       bit1 = 1 if station i was skipped
        int[][] choice = new int[N + 3][2];

        for (int i = N; i >= 1; i--) {
            // ── s = 0 (skip used) ──────────────────────────────
            if (dp[i + 1][0] >= dp[i + 2][0]) {
                dp[i][0]     = A[i] + dp[i + 1][0];
                choice[i][0] = 0;              // jump +1, collect
            } else {
                dp[i][0]     = A[i] + dp[i + 2][0];
                choice[i][0] = 1;              // jump +2, collect
            }

            // ── s = 1 (skip available) ─────────────────────────
            int best1 = dp[i + 1][1], best2 = dp[i + 2][1];
            int collectHere = A[i] + Math.max(best1, best2);
            int skipHere    = Math.max(dp[i + 1][0], dp[i + 2][0]);

            if (collectHere >= skipHere) {
                dp[i][1]     = collectHere;
                choice[i][1] = (best1 >= best2) ? 0 : 1;  // collect, jump
            } else {
                dp[i][1]     = skipHere;
                // mark bit1=2 to signal 'skip'; jump stored in lower bit
                boolean j1Better = dp[i+1][0] >= dp[i+2][0];
                choice[i][1] = (j1Better ? 0 : 1) | 2;    // skip flag
            }
        }

        // ── Trace path ─────────────────────────────────────────
        System.out.println("Maximum energy: " + dp[1][1]);
        StringBuilder path = new StringBuilder();
        int skippedStation = -1;
        int cur = 1, s = 1;

        while (cur <= N) {
            path.append(cur);
            int c    = choice[cur][s];
            boolean skipped = (c & 2) != 0;
            int     jump    = (c & 1) + 1;          // 1 or 2
```

```
            if (skipped) { skippedStation = cur; s = 0; }
            if (cur + jump <= N) path.append(" → ");
            cur += jump;
        }

        System.out.println("Path: " + path);
        if (skippedStation != -1)
            System.out.println("Skipped station: " + skippedStation
                               + "  (value=" + A[skippedStation] + ")");
        else
            System.out.println("No station skipped.");
    }
}
```

Verification — All Test Cases

Test Case 1

Input	Output
N=6 A=[3,-5,4,-2,7,1]	15

Optimal path: 1 → 3 → 5 → 6 | 3+4+7+1=15 | No skip needed.

Test Case 2

Input	Output
N=5 A=[1,-10,2,3,4]	10

Optimal path: 1 → 3 → 4 → 5 | 1+2+3+4=10 | No skip needed.

Test Case 3

Input	Output
N=7 A=[5,-8,6,-1,-2,10,-3]	21

Optimal path: 1 → 3 → 4 → 6 | 5+6+(−1)+10=20 | Skip station 4 (−1): 5+6+0+10=21.

Test Case 4

Input	Output
N=4 A=[-1,-2,-3,-4]	-1

All values negative. Robot must start at station 1 (−1), then jumps to 3 (skip: 0), then jumps past end. Collected: −1. Best achievable: −1.

#	Input Array	Answer	Skip Used?	Optimal Path
Test	Input Array	Answer	Skip Used?	Key Path
1	[3,-5,4,-2,7,1]	15	No	1→3→5→6
2	[1,-10,2,3,4]	10	No	1→3→4→5
3	[5,-8,6,-1,-2,10,-3]	21	Yes — station 4	1→3→4→6
4	[-1,-2,-3,-4]	-1	Yes — station 3	1→3

Appendix — Design Notes & Optimisations

A. Why Two DP Columns Suffice (Space Optimisation)

dp[i][s] only ever reads dp[i+1][*] and dp[i+2][*]. A rolling window of three rows (current, +1, +2) per skip state reduces space to O(1):

```
// Rolling 3-slot arrays  (index 0 = i+2, index 1 = i+1, index 2 = current)
int[] prev0 = {0, 0, 0};   // skip-used column
int[] prev1 = {0, 0, 0};   // skip-available column

for (int i = N; i >= 1; i--) {
    int d0 = A[i] + Math.max(prev0[1], prev0[0]);
    int d1 = Math.max(A[i] + Math.max(prev1[1], prev1[0]),
                              Math.max(prev0[1], prev0[0]));
    prev0[0] = prev0[1];  prev0[1] = d0;
    prev1[0] = prev1[1];  prev1[1] = d1;
}
return prev1[1];   // dp[1][1]
```

B. Comparison: D&C vs DP

Aspect	Divide & Conquer	Dynamic Programming
Time	O(2^N) — exponential	O(N) — linear
Space	O(N) — call stack	O(N) or O(1) rolling
Sub-problems	Recomputed many times	Each solved exactly once
Code length	Short & intuitive	Slightly longer but fast
Practical use	N ≤ ~20 only	N up to 10^6 easily

C. Edge Cases Handled

- N = 1: robot lands on station 1 and immediately exits. Answer = max(A[1], 0) — skip station 1 if negative.
- N = 2: robot visits station 1 then moves to 2 or exits. DP handles this with sentinel zeros.
- All negative: skip the most negative station visited; result is still negative but maximised.
- All positive: skip is never beneficial; dp[i][1] = dp[i][0] throughout.

Final Answer Formula

answer = dp[1][1]

Meaning: maximum energy starting at station 1 with the skip power still available.